



nextwork.org

Set Up a Web App Using AWS and VS Code

CH

Arpita Chowdhury

The screenshot shows the Visual Studio Code editor interface. The Explorer panel on the left displays the file structure of a project named 'nextwork-web-project'. The structure includes a 'src' directory with 'main', 'resources', and 'webapp' subdirectories. Inside 'webapp', there is a 'WEB-INF' directory containing 'index.jsp' and a 'pom.xml' file. The main editor window shows the content of 'index.jsp', which is an HTML file. The code in 'index.jsp' is as follows:

```
1 <html>
2
3 <body>
4
5   <h2>Hello Arpita!</h2>
6
7   <p>This is my NextWork web application working!</p>
8
9 </body>
10
11 </html>
```



Introducing Today's Project!

In this project, I will demonstrate launching an AWS EC2 instance, connecting to it securely using SSH via VS Code, installing Java and Maven, and generating a basic web application. I'm doing this project to learn how real-world remote development works on cloud servers, understand server-side environment setup, and experience manual code editing without an IDE highlighting why modern development tools are so essential in cloud engineering workflows.

This project is part one of a series of DevOps projects where I'm building a CI/CD pipeline. I'll be working on the next project in this series to automate builds, testing, and deployments using AWS DevOps services.

I did this project because I wanted hands-on experience building and managing a web application entirely in the cloud, while strengthening my understanding of EC2, SSH, and real-world remote development workflows.

Key tools and concepts

Services I used were Amazon EC2, AWS IAM, and AWS networking (SSH access), along with developer tools like Apache Maven, Amazon Corretto (Java), and VS Code Remote - SSH. Key concepts I learnt include launching and securing EC2 instances, key-pair authentication, remote server access via SSH, Java web app project generation with Maven, and remote development workflows using VS Code.

Project reflection

This project took me approximately 3 hours. The most challenging part was correctly setting up the SSH connection and handling file paths and permissions. It was most rewarding to successfully connect VS Code to my EC2 instance and see my Java web application created and edited directly on the cloud server.



Arpita Chowdhury

NextWork Student

nextwork.org

One thing I didn't expect in this project was how much simpler and more efficient remote development feels when using VS Code compared to editing files directly in the terminal.

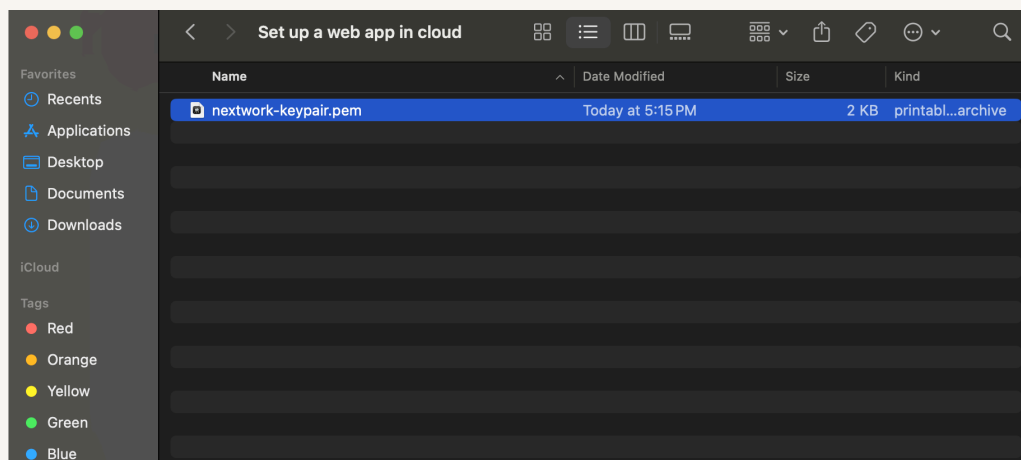


Launching an EC2 instance

What I did in this step

In this step, I will launch an EC2 instance because I want to set up key pair to secure access and network settings for my instance.

I started this project by launching an EC2 instance because it provides a secure, cloud-based virtual server where I can build, run, and manage my web application entirely in AWS, just like in real-world cloud environments.





I also enabled SSH

SSH is a secure protocol used to connect to and manage remote servers. I enabled SSH so that I could securely access and control my EC2 instance from my local machine.

Key pairs

A key pair is a set of cryptographic credentials (a public key and a private key) used to securely authenticate and access an EC2 instance via SSH without using passwords.

Downloaded key pair file

Once I set up my key pair, AWS automatically downloaded the private key file (.pem), which I use to securely connect to my EC2 instance via SSH.



Set up VS Code

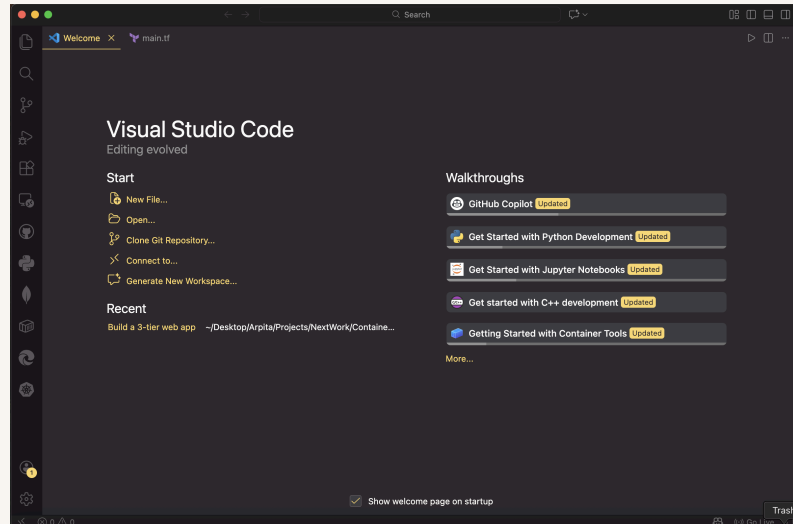
What I did in this step

In this step, I will install VS Code, set up a terminal to communicate with my EC2 instance, and update my key pair's permission settings because secure remote access and proper tooling are essential for managing and developing applications on cloud servers.

What is VS Code?

VS Code is a lightweight, powerful code editor that provides an integrated terminal, extensions, and remote development features for writing and managing code efficiently.

I installed VS Code to securely connect to my EC2 instance, run terminal commands, and manage my application files efficiently during remote development.





My first terminal commands

A terminal is a command-line interface that allows you to interact directly with your computer or a remote server by running text-based commands. The first command I ran for this project was `ls Desktop/Arpita/Projects/NextWork`, which I used to view the available directories and files before navigating to the project folder.

Updating file permissions

I also updated my private key's permissions by running `chmod 400 nextwork-keypair.pem`, which restricts access to the key file so only the owner can read it—ensuring secure SSH authentication with my EC2 instance.

```
arpitachowdhury@Arpita-MacBook-Air ~ % cd Desktop/Arpita/Projects
arpitachowdhury@Arpita-MacBook-Air Projects % cd NextWork
arpitachowdhury@Arpita-MacBook-Air NextWork % ls
AI_on_Cloud      AWS-console-home.png  AWS-Database      Basics             Cloud-Computing.png  Containers_and_Compute  DevOps             Invoice-DRBT9K9A-0001.pdf  Networking
Receipt-2212-2312.pdf  Security
arpitachowdhury@Arpita-MacBook-Air NextWork % cd Containers_and_Compute
arpitachowdhury@Arpita-MacBook-Air Containers_and_Compute % ls
APIs with Lambda + API Gateway  Deploy an app with docker  Fetch Data with AWS Lambda
Build a 3-tier web app          Deploy Backend with Kubernetes  Launch a Kubernetes Cluster
Create Kubernetes Manifests     Deploy an app across accounts  Set up Kubernetes Deployment
arpitachowdhury@Arpita-MacBook-Air Containers_and_Compute % cd .
arpitachowdhury@Arpita-MacBook-Air Containers_and_Compute % cd .
zsh: command not found: cd.
arpitachowdhury@Arpita-MacBook-Air Containers_and_Compute % cd .
arpitachowdhury@Arpita-MacBook-Air Containers_and_Compute % cd ..
arpitachowdhury@Arpita-MacBook-Air NextWork % cd DevOps
arpitachowdhury@Arpita-MacBook-Air DevOps % ls
Build a CI/CD Pipeline with AWS      Create S3 Buckets with Terraform  Multi-Cloud Data Transfer with AWS and GCP
Connect a GitHub Repo with AWS       Deploy a Web App with CodeDeploy  Secure Packages with CodeArtifact
Continuous Integration with CodeBuild  Infrastructure as Code with CloudFormation  Set up a web app in cloud
arpitachowdhury@Arpita-MacBook-Air DevOps % cd Set up a web app in cloud
cd: too many arguments
arpitachowdhury@Arpita-MacBook-Air DevOps % cd Set up a web app in cloud
cd: too many arguments
arpitachowdhury@Arpita-MacBook-Air DevOps % cd Set\ up\ a\ web\ app\ in\ cloud
arpitachowdhury@Arpita-MacBook-Air Set up a web app in cloud % chmod 400 nextwork-keypair.pem
arpitachowdhury@Arpita-MacBook-Air Set up a web app in cloud %
```



SSH connection to EC2 instance

What I did in this step

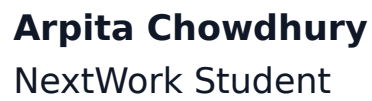
In this step, I will connect to my EC2 instance using SSH from VS Code because establishing a secure remote connection allows me to work directly on the cloud server where my web application will be built and run.

Connecting to EC2

To connect to my EC2 instance, I ran the command `ssh -i nextwork-keypair.pem ec2-user@ec2-54-221-144-104.compute-1.amazonaws.com`, which securely authenticated me using my key pair and opened a remote SSH session on the server.

This command required an IPv4 address

A server's IPv4 DNS is a public hostname that maps to the server's IPv4 address, allowing you to connect to the server using a readable domain name instead of a numeric IP address.

[illegible]



Maven & Java

What I did in this step

In this step, I will install Apache Maven and Amazon Corretto 8 on my EC2 instance and verify the installations because these tools are required to build, manage, and run a Java-based web application directly on the cloud server.

Why I'm using Maven

Apache Maven is a build automation and dependency management tool used to create, manage, and package Java projects in a standardized and efficient way.

Maven is required in this project because it automates the process of building the Java web application, manages project dependencies, and ensures the application can be compiled and packaged consistently on the EC2 instance.

Why I'm using Java

Java is a widely used, platform-independent programming language that enables developers to build, run, and deploy scalable applications across different operating systems.

Java is required in this project because the web application is built using Java, and the EC2 instance needs a Java runtime to compile, run, and execute the application.



Create the Application

What I did in this step

In this step, I will run Maven commands in my EC2 terminal to generate a starter Java web application because Maven can automatically create the project structure and files needed to begin building and running a web app quickly and correctly.

Creating the Java web app

I generated a Java web app using the command `mvn archetype:generate -DgroupId=com.nextwork.app -DartifactId=nextwork-web-project -DarchetypeArtifactId=maven-archetype-webapp -DinteractiveMode=false`, which automatically created a standard Maven-based web application structure.

Installing Remote - SSH

I installed Remote - SSH, which is a VS Code extension that allows me to connect to and work on a remote server over SSH using the full VS Code interface. I installed it to easily view, edit, and manage my web application files directly on my EC2 instance without relying only on the terminal.



SSH configuration details

Configuration details required to set up a remote connection include the EC2 instance's public IPv4 DNS, the SSH username (such as ec2-user), and the path to the private key (.pem) file used for secure authentication.

```
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/4/maven-parent-4.pom
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/4/maven-parent-4.pom (10.0 kB at 182 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/apache/3/apache-3.pom
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/apache/3/apache-3.pom (3.4 kB at 45 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/archetypes/maven-archetype-webapp/1.0/maven-archetype-webapp-1.0.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/archetypes/maven-archetype-webapp/1.0/maven-archetype-webapp-1.0.jar (3.9 kB at 54 kB/s)
[INFO]
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-webapp:1.0
[INFO]
[INFO] Parameter: basedir, Value: /home/ec2-user
[INFO] Parameter: package, Value: com.nextwork.app
[INFO] Parameter: groupId, Value: com.nextwork.app
[INFO] Parameter: artifactId, Value: nextwork-web-project
[INFO] Parameter: packageName, Value: com.nextwork.app
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/ec2-user/nextwork-web-project
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO]
[INFO] Total time: 19.094 s
[INFO] Finished at: 2020-01-03T23:47:29Z
[INFO] Final Memory: 19M/87M
[INFO]
[ec2-user@ip-172-31-23-93 ~]$
```

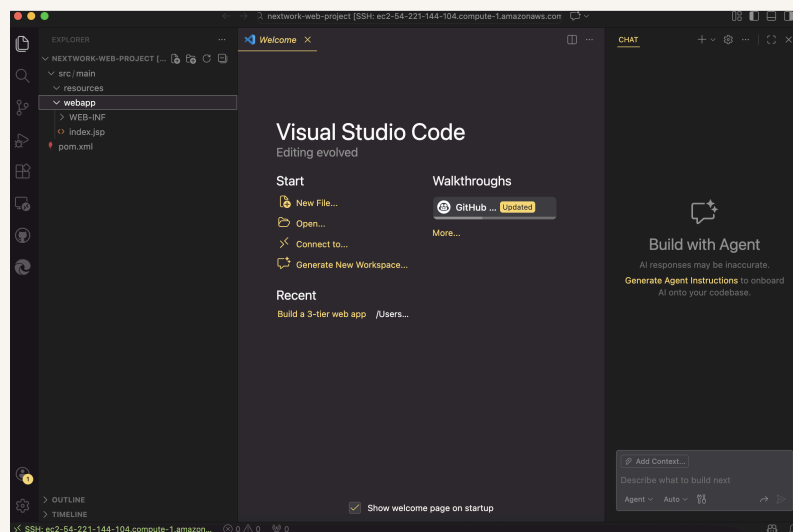


Create the Application

Exploring the project structure

Using VS Code's file explorer, I could see the full nextwork-web-project directory structure on my EC2 instance, including the src/main/webapp folder, the WEB-INF directory, the index.jsp file, and the pom.xml file.

Two of the project folders created by Maven are src and webapp, which store the application's source code and the web-facing files (such as JSP pages and configuration resources) used to run the Java web application.





Using Remote - SSH

What I did in this step

In this step, I will install the VS Code Remote SSH extension and use it to connect VS Code directly to my EC2 instance because this allows me to visually explore, edit, and manage my Java web application files on the cloud server using a full IDE instead of only the terminal.

Updating the web app

The `index.jsp` is the main entry page of the Java web application, where JSP/HTML code defines what users see when the web app is accessed in a browser.

I edited `index.jsp` by updating the HTML content to display a custom message, allowing me to personalize the web page and confirm that my changes were reflected in the Java web application.



The screenshot shows an IDE window titled "nextwork-web-project [SSH: ec2-54-221-144-104.compute-1.amazonaws.com]". The left sidebar, labeled "EXPLORER", displays the project structure: "NEXTWORK-WEB-PROJECT [SSH: EC2-54-221-144-104.compute-1.amazonaws.com]" containing "src/main", "resources", "webapp", "WEB-INF", "index.jsp", and "pom.xml". The main editor area shows the "index.jsp" file with the following HTML code:

```
1 <html>
2
3 <body>
4
5   <h2>Hello Arpita!</h2>
6
7   <p>This is my NextWork web application working!</p>
8
9 </body>
10
11 </html>
```



Using nano

Additional improvements

In this secret mission, I will edit `index.jsp` directly using the terminal instead of an IDE because it helps me understand low-level file editing on a remote server and appreciate the efficiency and convenience that IDEs like VS Code provide.

Terminal vs IDE

An alternative to using IDEs is editing files directly through the terminal using command-line text editors like nano. To edit `index.jsp`, I ran the command `nano index.jsp`, which allowed me to modify the file from within the EC2 instance.

Compared to using an IDE, editing `index.jsp` in the terminal felt slower and less intuitive, with limited visual feedback and tooling. I'd be more likely to use an IDE if I want faster editing, syntax highlighting, error detection, and an overall more efficient development experience.

Verifying my work

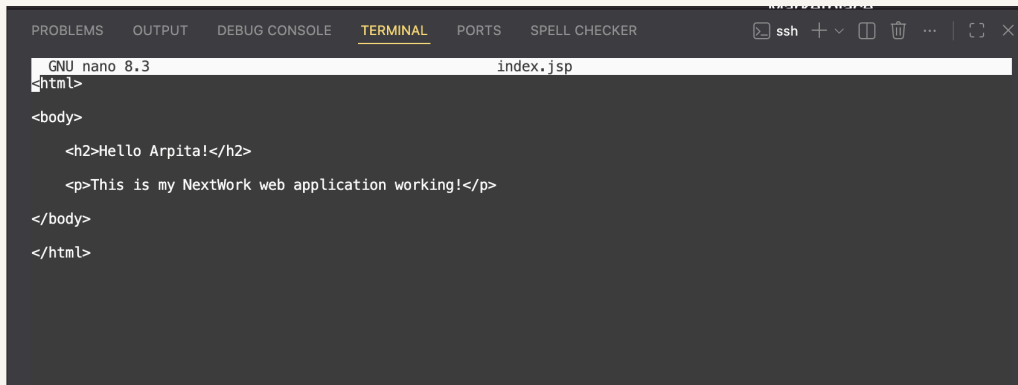
To verify my editing work in the terminal, I saved the file and checked the updated content directly in VS Code. It was possible to see my changes in VS Code right away because both the terminal and VS Code were connected to the same EC2 instance and working on the same remote files in real time.



Arpita Chowdhury

NextWork Student

nextwork.org



The screenshot shows a code editor window with a dark theme. The top bar includes tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (which is active), PORTS, and SPELL CHECKER. On the right side of the top bar, there are icons for SSH, a plus sign, a minus sign, a trash can, and a close button. The main editing area shows the content of a file named 'index.jsp'. The code is as follows:

```
GNU nano 8.3 index.jsp
<html>

<body>

  <h2>Hello Arpita!</h2>

  <p>This is my NextWork web application working!</p>

</body>
</html>
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

